COMPLETE ENGINEERING CHANGE REPORT

Distributed Recommender Simulator

Full before-and-after documentation of Retail50K fixes, Doc2Vec batch mode, experiment reset, GUI modernization, recommendation target semantics, graph control, runtime reliability, scalable data generation, memory scheduling, Exact Multi-Node Lab, journal-dataset findings, and Doc2Vec cache transparency.

Field	Value
Prepared for	Mohsen Alavi
Academic context	DSMP / Distributed Recommender Simulator
Primary repository	github.com/aabhari/test
Current working folder	/Users/mohsen11/Desktop/June 10 session final/test
Included prior attachment	/Users/mohsen11/Desktop/abhari june 9 user sim try/test/DSMP_Retail50K_Full_Report.pdf
Report date	16 June 2026, America/Toronto
Status	Code-grounded documentation with current compile and focused self-test verification

REPOSITORY STATE WARNING The local tree is intentionally rich with source changes, generated datasets, caches, result logs, rendered reports, and screenshots. This paper documents the engineering state and behavior; release work still needs a clean source-only commit series.

Abstract

This report documents the full DSMP simulator workstream represented by the attached Retail50K report and the current June 10-16 development conversation. The effort began as dataset-specific debugging and grew into a broader correctness, usability, scalability, and observability modernization. The guiding rule was to preserve the meaning of the existing algorithms when `Algorithm Default` is selected, while exposing previously hidden semantics and preventing known false or partial recommendation outputs.

The most important technical corrections are: Retail50K names with spaces now survive message parsing and Weka ARFF serialization; Retail graph files are dataset-scoped; SVM uses class-preserving splits and calibrated class distributions; Doc2Vec batch mode computes every target independently; graph rendering can be skipped while edge state is still persisted; a dedicated reset creates a clean experiment boundary; repeated-run lifecycle IDs block stale messages; idle agents block instead of polling; the user-generation engine can atomically create validated large corpora; and high-memory model work is coordinated by heap-aware scheduling.

The later work also adds a modern Swing GUI, searchable file selection, recommendation result cards, explicit recommendation target modes, an Exact Multi-Node Population Lab, real-time node activity dashboards, configurable Java heap restart, and Doc2Vec cache transparency in both the Simulator and Exact Lab tabs. The implementation is materially stronger, but not magic: Exact Weka SVM remains expensive on huge feature spaces, Doc2Vec cold training is still real work, and converted recommendation modes must be reported as converted or hybrid objectives rather than untouched native algorithms.

DEFINITE SUMMARY The current local version is more correct, more observable, and more reliable than the starting version. The safest academic statement is that Algorithm Default preserves native recommendation semantics where the implementation was not deliberately corrected for a bug, while new modes and labs are explicit additions with clearly labeled behavior.

Executive Change Inventory

Area	When	Before	Now
Retail50K parser and ARFF fixes	Older bug	Names such as `T&T Supermarket` were split by spaces.	Tab protocol and ARFF quoting keep the class intact.
Retail graph isolation	Older bug	Graph mapping/edge files were missing or shared across datasets.	Dataset-specific name-number, edge, and GML artifacts.
SVM split and score calibration	Older bug	Small classes could lose all training examples; scores were hard 1/0.	Stratified class splits and calibrated SMO distributions.
Doc2Vec batch	Chat change	Multiple selected targets received the first user's score map.	Each target is scored independently using the shared scorer.
Reset Experiment	Chat change	Cleanup was tied to Get Users or app lifecycle.	Dedicated reset preserves inputs/caches and removes transient experiment state.
GUI modernization	Chat change	Dense legacy form, raw output, fragile dialogs.	Modern panels, cards, status, search picker, and recommendation cards.
Target modes	Chat change	Algorithms returned users or accounts without explaining the entity type.	`Algorithm Default`, `Similar Users`, and `Followee Accounts` are explicit.

Graph checkbox	Chat change	Recommendation completion always went through Gephi output.	Graph display/export can be skipped while edges still accumulate.
Idle CPU cleanup	Chat change	Agents kept polling and Gephi project stayed open.	Blocking JADE message handling, async completion, Gephi cleanup.
Large corpus generation	Chat change	Million-row generation was not atomic or validated.	Streaming exact-count generator with schema validation.
Streaming and sparse data path	Chat change	Huge datasets duplicated rows/maps and dense ARFF data.	Replayable streams, sparse rows, indexing, reduced live memory.
Adaptive memory	Chat change	All nodes could overload the JVM.	Heap-aware leases, safe waves, worker scheduling, heap GUI.
Exact Multi-Node Lab	Chat change	Multi-node effect was hard to prove without changing graph state.	Isolated target sweeps with one-worker equivalence verification.
Doc2Vec cache UI	Latest change	Fast cached Doc2Vec looked suspicious and unexplained.	Warm/cold/stale cache status, rows/users/vectors/key/size, scoped remove button.

1. Scope, Evidence, and Method

The report combines two bodies of work. The first is the attached June 7 Retail50K report, which records pre-chat fixes for dataset parsing, Retail graph files, timing instrumentation, v2.5 compatibility, multi-node K-Means measurements, SVM batch observations, shared Doc2Vec cache work, and the final SVM root-cause fix. The second is the current chat work, which extends the simulator with corrected Doc2Vec batch mode, reset controls, GUI modernization, recommendation target modes, graph skip controls, runtime lifecycle hardening, million-row generation and ingestion, exact/isolated population sweeps, journal-dataset investigation, and Doc2Vec cache transparency.

Evidence was gathered from the extracted prior PDF, the June 13 engineering report already present in `reports/`, current source-code inspection, current compile output, focused self-test execution, generated Swing panel renders, and user-provided screenshots captured during actual runs. The report distinguishes source-code changes from analysis-only conclusions and communication artifacts.

Evidence source	What it contributes	Limit
Attached Retail50K PDF	Older fixes, timings, and SVM root-cause analysis.	Line numbers refer to an older folder and may not match the current tree.
Current source tree	Actual implementation state after the latest cache UI additions.	Worktree is dirty and includes generated artifacts.
Current compile	All current production Java source compiles against `lib/*`.	Compile is not the same as a full end-to-end GUI run.
Focused self-tests	Doc2Vec batch parsing, recommendation merging, post-processing, heap settings, rendered UI panels.	Self-tests cover targeted invariants, not every dataset.
Screenshots/renders	Visual evidence of the GUI surfaces discussed.	Some are development screenshots; generated renders are labeled as renders.

VERIFICATION PERFORMED FOR THIS REPORT Current production Java compilation passed. PopulationSweepPanelSelfTest, NodeExecutionDashboardSelfTest, RecommendationScoreMergerSelfTest, RecommendationPostProcessorSelfTest, RuntimeResourceSettingsSelfTest, and Doc2VecBatchRunnerSelfTest passed. The Doc2Vec self-test emitted the known Gensim compatibility diagnostic but still passed.

2. Prior Retail50K Fixes Integrated from the Attached PDF

The attached report is not treated as a separate appendix. Its important engineering points are integrated here in the same before/problem/change/result/risk structure used for the rest of this paper.

2.1 Retail50K MLP Crash: Space-delimited Message Protocol

Before: MobileAgent sent tweet data as one space-delimited string, and RecommenderAgent split it by spaces.

Why bad: `T&T Supermarket` became `T&T`, which no longer matched the class label built from the original dataset. MLP then looked up a missing class and could crash.

Change: the active path changed tweet messages to tab-separated fields and added a manual tab parser with a legacy fallback.

Now: names with spaces survive the agent protocol. The parser preserves `T&T Supermarket` as one followee/class field.

Risk: legacy messages can still enter through fallback paths; tests should keep a space-containing class fixture.

2.2 Uploaded Dataset Path and Retail Graph Mapping

Before: after a user selected `/Users/mohsen11/Downloads/Retail50K.txt`, later code sometimes looked for `important-stuff/Retail50K.txt`. Retail graph generation also assumed `name-number-Retail50K.txt` already existed.

Change: active MobileAgent and graph code use the selected file object. The program creates the missing name-number mapping from the selected dataset and writes dataset-specific edge/GML files such as `edges-numbers-Retail50K.txt`.

Result: the prior report observed Retail50K mapping at 628 unique graph nodes, with edges accumulating by run while node count remained stable. This fixed old graph contamination and missing-file failures.

Risk: graph node count is now dataset-specific, but stale artifacts can still confuse results if a user intentionally mixes datasets without Reset Experiment. The new reset button later addresses that workflow risk.

2.3 Weka ARFF Quoting for Class Names

Before: ARFF class values were written raw. Weka could display `T&T Supermarket` as separate `T&T` and `Supermarket` confusion-matrix labels.

Change: ARFF class values are quoted and escaped when the class attribute and row labels are written. Backslashes, quotes, and line breaks are normalized before serialization.

Now: Weka receives the intended single class value. This preserves the SVM/MLP classifier family but fixes invalid file syntax.

2.4 Version 2.5 Compatibility Work

The prior report also records changes in the older `/Users/mohsen11/Desktop/abhariv2.5/SocialNetworkSimulatorV3` folder: the same tab-delimited Retail fix, SVM ARFF quoting, replacement of a hardcoded Windows Python path with OS-sensitive `python`/`python3`, and a graph-import guard when an expected GML file is missing.

These changes are important historically, but they are not the main current workspace. They show that the Retail50K issue was not one isolated symptom; it exposed file-format, launcher, and graph assumptions across versions.

2.5 Stage Timers, Tweet Batching, and K-Means Load Balancing

Before: the simulator sent each tweet as a separate JADE ACL message. Timing was hard to interpret because fixed waits, graph layout, text processing, TF-IDF, and model work were mixed together.

Change: stage timers were added; text-file paths for non-MLP algorithms batch a user's tweets into one serialized message; text preprocessing uses precompiled patterns and a single tokenization path; and text-node assignment estimates load from tweet count, unique terms, and character count.

Result: the prior report observed K-Means recommender-stage improvement on 8 nodes, for example reducing the algorithm/TF-IDF stage from about 38.3 s on one node to about 15.3 s on eight nodes in the reported Retail50K run. The total GUI wall time still remained dominated by fixed MobileAgent waiting and Gephi layout until later changes removed or made parts optional.

Risk: stage timings depend on dataset, JVM, machine, and algorithm. They should be reported with node count, heap, graph setting, and dataset checksum rather than reused as universal numbers.

2.6 SVM Root-Cause Fix for Low Retail Diversity

The attached PDF's final and most important Retail SVM finding is that SVM trains on aggregated user documents, not on raw rows. A row-balanced Retail50K_20x file can still be user-imbalanced because Wal-Mart may have hundreds of user documents while `T&T Supermarket` has only a few.

Before: the train/test split filled the test set first by repeatedly removing users from each class. Small retailer classes could be completely consumed by the test set, leaving zero training examples. SVM scores were also hard 1/0 values from a single `classifyInstance()` prediction.

Change: SVM splitting became stratified inside each class, keeping training examples for minority classes when possible. Weka SMO still remains the classifier, but calibration is enabled and `distributionForInstance()` is converted into a full score map.

Now: SVM recommendation scores are class distributions rather than one predicted class turned into 100/0. Batch SVM uses the same distribution path. This improves recommendation meaning while preserving the TF-IDF -> Weka SMO pipeline.

Risk: calibration and stratification deliberately change SVM output compared with the pre-fix bug. This is a correctness repair, not a neutral performance refactor. It should be named in any comparison with older branches.

3. Doc2Vec Batch Mode and Cache Transparency

3.1 Correct Multi-target Batch Mode

Before: the Java Doc2Vec branch built a list of selected users but invoked Python with only the first selected target. The returned score map was then assigned to every selected target. Enabling the old SVM batch field for Doc2Vec would have been silently wrong.

Change: `Doc2VecBatchRunner.java` and `doc2vec_batch.py` implement a structured batch path. Java writes the targets to a temporary file; each recommender node starts one Python process; Python loads or trains the shared Doc2Vec model once and calls the same `score_candidate_shard()` function independently for every target.

Now: every target receives its own score map. Candidate sharding remains node-specific and the organizer still merges per-target scores through the existing flow. Per-user errors are explicit and do not become fake successful recommendations.

Invariant	How the implementation preserves it
Same scorer	Batch uses the existing `doc2vec_shared.score_candidate_shard()` logic.
Same cache signature	Model reuse is keyed by dataset path, size, modification time, parameters, workers, and script/kernel version.
Same shard concept	Each recommender node scores only its assigned candidate shard.

Independent target maps	Structured `DOC2VEC_SCORE` records include target and candidate identity.
Failure visibility	Missing users, Python errors, or unsafe numerical diagnostics create explicit failures or empty maps.

3.2 Why 618 Doc2Vec Targets Can Finish in About One Second

The fast Exact Lab result is valid only under the condition the GUI now exposes: a warm shared Doc2Vec cache already exists. In that case the program is not retraining Doc2Vec. It is loading cached normalized vectors and scoring target vectors against candidate vectors. For a dataset with hundreds of users, this can be very fast on modern hardware.

This was the exact reason the UI needed cache transparency. A one-second run looks suspicious if the GUI just says Doc2Vec, but it is understandable if the GUI says `Warm cache used`, `Training skipped`, `618 user vectors`, and displays the cache key.

> **IMPORTANT INTERPRETATION** A warm-cache Doc2Vec run is a scoring-only run. A cold-cache run includes model training and can be much slower. The recommendation logic is not skipped; the training stage is skipped because the matching model is already on disk.

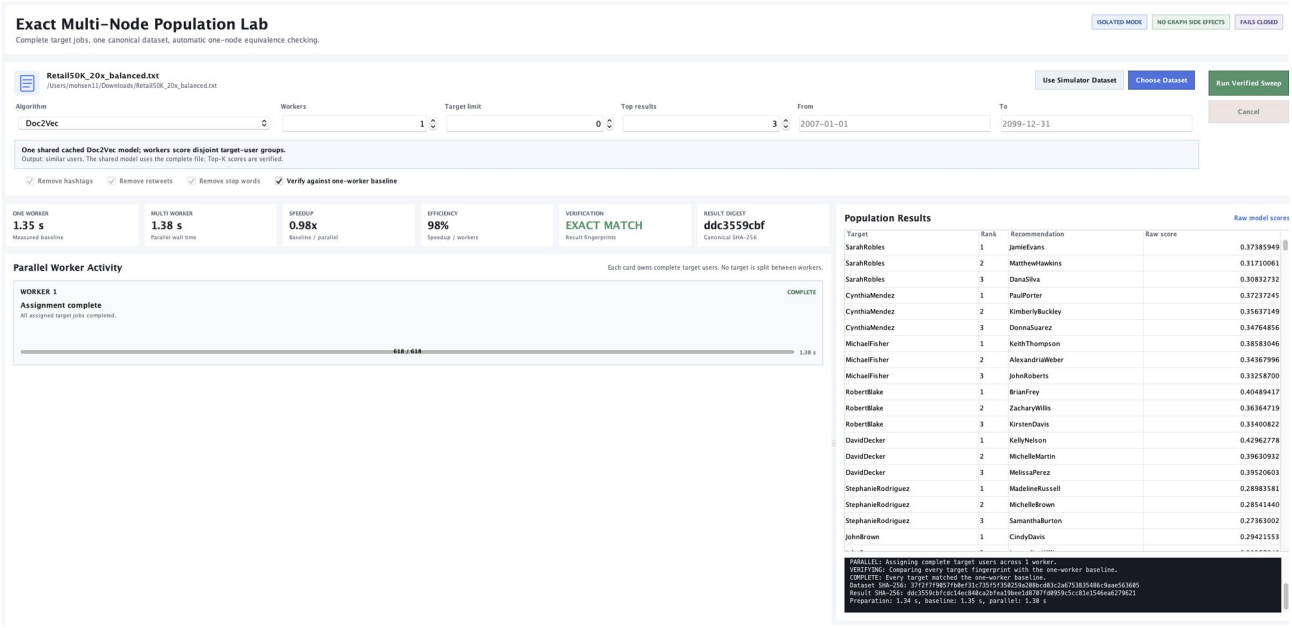


Figure 1. Development screenshot of the Exact Multi-Node Lab after a warm Doc2Vec cache run. The table shows 618 target results and exact one-worker verification.

3.3 Cache Transparency in Both Tabs

Latest change: the cache status panel was added to the Exact Multi-Node Lab and then to the main Simulator tab whenever Doc2Vec is selected. The UI inspects the selected dataset through `Doc2VecCacheAdmin.java` and `doc2vec_cache_admin.py`.

Displayed field	Meaning
State	`WARM`, `COLD`, or `STALE` based on whether a valid matching cache exists.
Rows	Dataset row count read by the same cache-inspection path.

Users	Number of distinct Doc2Vec document tags/users.
Vectors	Count and dimensionality of cached scoring vectors when available.
Cache size	Exact matching cache size and dataset-related cache size.
Cache key	Shortened shared cache key, with full key available as tooltip.
Refresh Cache	Re-runs inspection without starting a recommendation run.
Remove This Cache	Deletes only cache files tied to the selected dataset and blocks deletion during an active simulator run.

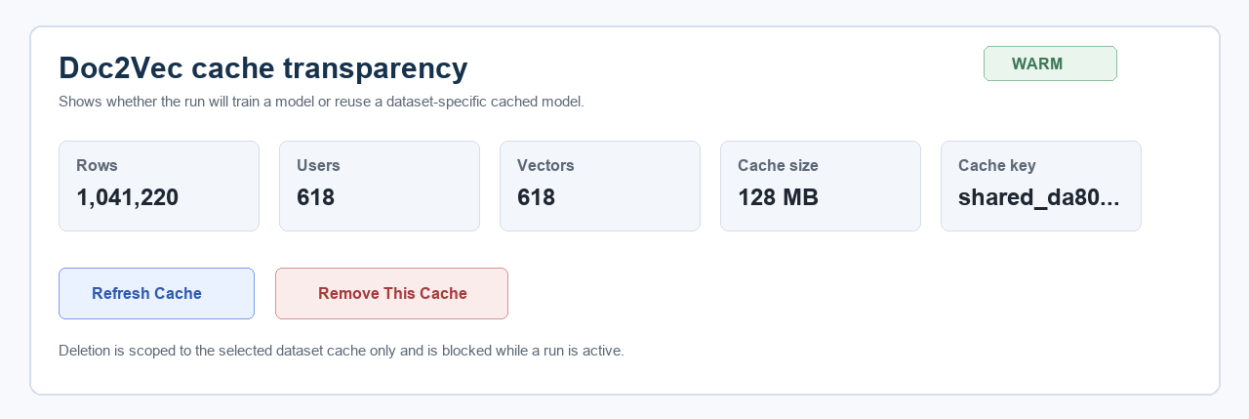


Figure 2. Rendered evidence panel for the Doc2Vec cache transparency card added to the main Simulator and Exact Lab workflows.

Risk: deleting the cache does not delete the dataset. It forces a future Doc2Vec cold train. If another Python process is actively training or scoring, cache deletion is intentionally disabled from the Simulator UI to avoid corrupting an active run.

4. GUI, Reset, Recommendation Modes, and Graph Control

4.1 Dedicated Reset Experiment

Before: cleanup was tied to closing the GUI or clicking Get Users. That made it unclear whether the user was merely loading users or destroying the current experiment's transient graph and recommendation state.

Change: `Reset Experiment` became an explicit GUI command. It stops active agents, clears in-memory state, resets progress and selections, and deletes known transient experiment artifacts while preserving input datasets, archived Results folders, and reusable Doc2Vec caches.

Now: users can intentionally start a brand-new experiment without closing the app and without coupling cleanup to `Get Users`. The reset confirmation explains that current graph/recommendation-side state is discarded.

4.2 Modern Simulator Workspace

The old UI was a dense form with raw console output. The modernized UI uses a left data-source/user rail, a central experiment setup workspace, and a right live-output panel. The app title is left aligned, redundant colored badges were removed, actions are grouped in the header, and status is presented as a labeled progress message rather than a vague text area.

The data-source panel now uses source cards for text file, database, generated, and artificial datasets. The Experiment Setup panel is organized by model/objective, workload, runtime resources, data window, text processing, post-processing, and run pipeline. The recommendation output view now renders cards with target identity, rank, candidate, score, and candidate detail instead of raw text.

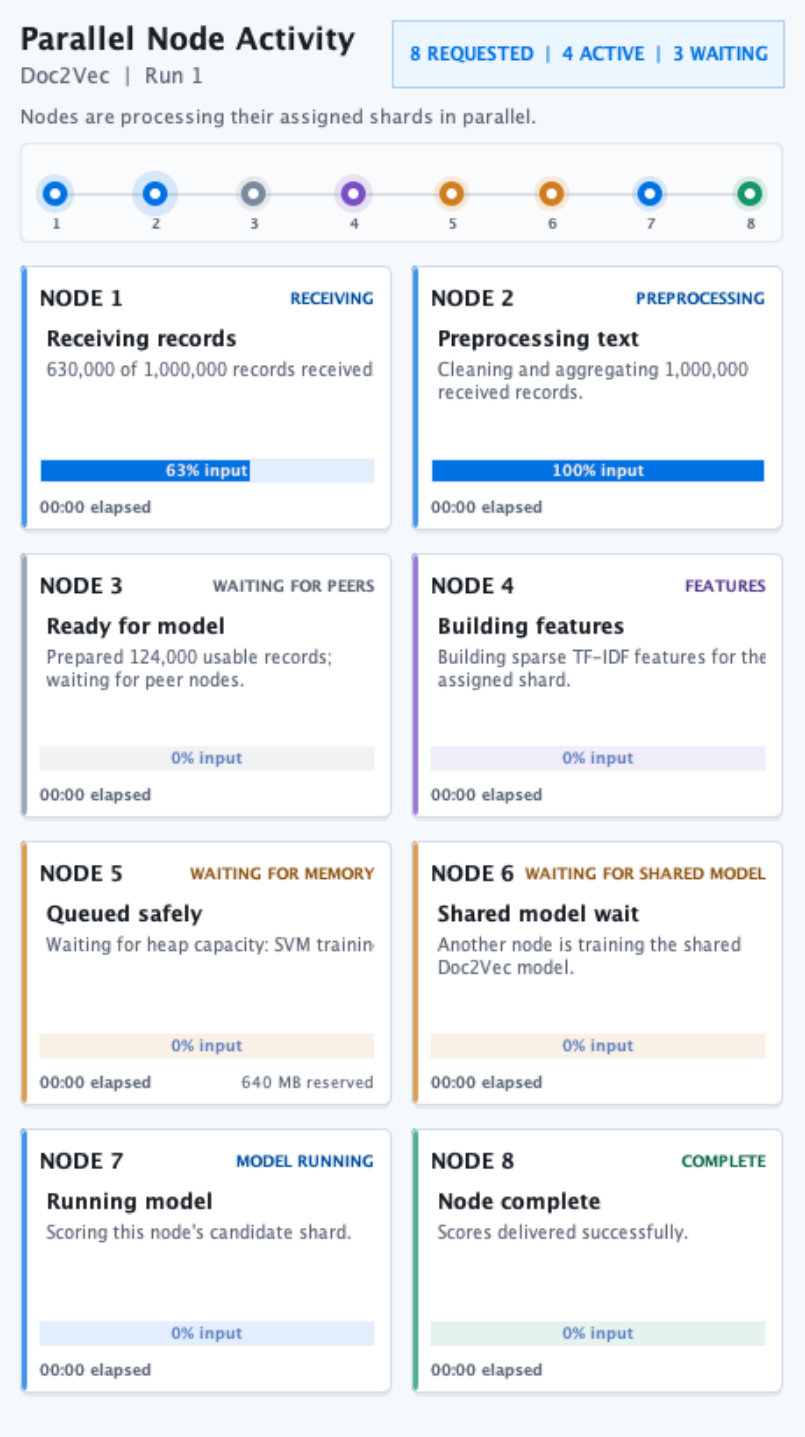

Parallel Node Activity
Doc2Vec | Run 1
8 REQUESTED | 4 ACTIVE | 3 WAITING
Nodes are processing their assigned shards in parallel.
1
2
3
4
5
6
7
8
NODE 1
RECEIVING
Receiving records
630,000 of 1,000,000 records received
63% input
00:00 elapsed
NODE 2
PREPROCESSING
Preprocessing text
Cleaning and aggregating 1,000,000 received records.
100% input
00:00 elapsed
NODE 3
WAITING FOR PEERS
Ready for model
Prepared 124,000 usable records; waiting for peer nodes.
0% input
00:00 elapsed
NODE 4
FEATURES
Building features
Building sparse TF-IDF features for the assigned shard.
0% input
00:00 elapsed
NODE 5
WAITING FOR MEMORY
Queued safely
Waiting for heap capacity: SVM trainin
0% input
00:00 elapsed
640 MB reserved
NODE 6
WAITING FOR SHARED MODEL
Shared model wait
Another node is training the shared Doc2Vec model.
0% input
00:00 elapsed
NODE 7
MODEL RUNNING
Running model
Scoring this node's candidate shard.
0% input
00:00 elapsed
NODE 8
COMPLETE
Node complete
Scores delivered successfully.
0% input
00:00 elapsed

Figure 3. Rendered node activity dashboard. The UI can show each requested node's state, received input, waiting stage, and model work.

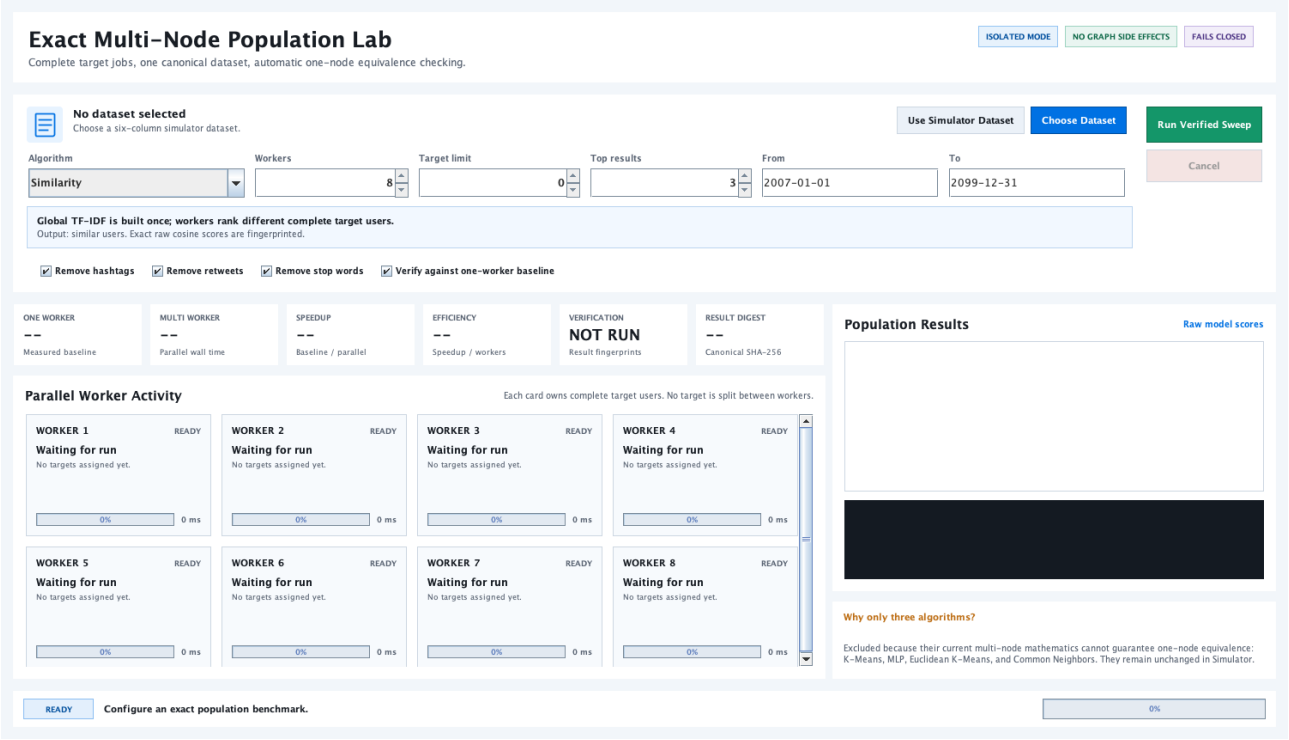


Figure 4. Rendered Exact Multi-Node Population Lab panel used to verify the new lab surface can paint correctly in headless tests.

4.3 File Picker Repair and Search

Before: the native Swing file chooser could crash on macOS with `IllegalComponentStateException: The dialog is decorated`, and it offered little help for finding large datasets across project folders.

Change: `ModernFilePicker.java` provides an application-owned file picker with search/filtering and safer asynchronous behavior. This avoids the decorated native dialog failure and better matches the redesigned UI.

4.4 Recommendation Target Modes

Before: the program silently mixed recommendation entity types. Similarity, K-Means, K-Means Euclidean, and Doc2Vec naturally ranked other users. SVM and MLP naturally predicted/ranked field-0 followee accounts. That is why Retail SVM showed retailer names while Doc2Vec showed people.

Change: the GUI adds `Recommendation target`: `Algorithm Default`, `Similar Users`, or `Followee Accounts`. Supporting classes such as `RecommendationTargetMode`, `RecommendationCandidateType`, `RecommendationModeSupport`, `RecommendationScorePayload`, and `RecommendationScoreMerger` make the entity type explicit and serializable.

Algorithm	Algorithm Default	Similar Users	Followee Accounts
Similarity	Ranks users by cosine similarity.	Same native output.	Maps ranked users to their field-0 accounts.
K-Means	Ranks users through the legacy cluster path.	Same native output.	Collapses similar users to account recommendations.
K-Means Euclidean	Ranks users through Euclidean cluster scoring.	Same native output.	Maps ranked users to accounts.

Doc2Vec	Ranks users by document-vector similarity.	Same native output.	Maps similar users to their accounts, showing the representative user detail.
SVM	Ranks field-0 accounts from Weka SMO distribution.	Hybrid: uses classifier signal and user evidence to rank users.	Same native output.
MLP	Ranks field-0 accounts from neural classifier output.	Hybrid: classifier-guided user ranking.	Same native output.

RESEARCH REPORTING RULE Only Algorithm Default should be described as the native algorithm answer. The other two options are useful and implemented deliberately, but they are selected recommendation objectives and must be reported as such.

4.5 Optional Graph Output with Persistent Edges

Before: after recommendations were produced, the simulator always entered the Gephi visualization/export phase. This made runs feel stuck when the user only wanted the recommendations.

Change: a `Generate Final Graph Output` checkbox was added. When unchecked, the simulator skips Gephi display/export, but still appends the new recommendation edge and updates the dataset-specific graph state.

Now: a user can run 10 targets with graph output off, then turn graph output on for the 11th target and receive a graph containing the previous 10 targets' accumulated edges. The graph checkbox controls visualization/export, not whether recommendations are persisted.

4.6 User Generation Studio Attribution and Header

The User Generation Studio header was simplified by removing the colorful mode boxes under the title and aligning the title to the left. The page now places `DSMP Lab: Dr. Abdolreza Abhari` and `Development support: Mohsen Alavi` in a compact academic attribution zone rather than as detached decorative text.



Figure 5. User-provided before screenshot that motivated removal of the colored badges and left-aligned studio header.

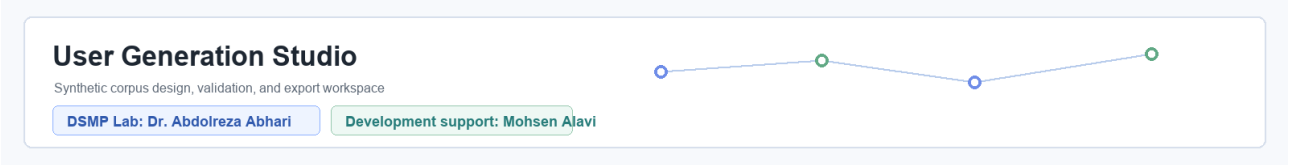


Figure 6. Rendered target header design showing the aligned title and academic attribution placement.

5. Runtime Reliability, Idle CPU, and Post-processing

5.1 Blocking JADE Message Delivery and Run Lifecycle

Before: several agents polled frequently or could consume stale messages from a previous run. This caused high CPU usage after a run appeared finished and created races where score messages could be consumed by the wrong handler.

Change: idle handlers now use blocking JADE delivery or explicit message templates. `RecommendationRunLifecycle` assigns UUID-backed run identities so agents can distinguish a new run, the current run, and retired run messages. Stale score/completion messages are ignored instead of participating in a new merge.

Now: the application should not sit at extreme CPU usage merely because the GUI remains open. Completion popups are asynchronous, and StarterAgent closes the embedded Gephi project after graph export.

5.2 Fast Post-recommendation Persistence

Before: after recommendations appeared, MobileAgent could still scan the dataset seven to nine times and rewrite a full `Dataset-Test-*` file. On a million-row file this created minutes of apparent delay even with graph output disabled.

Change: `RecommendationDatasetContext` and `RecommendationPostProcessor` use the already-built dataset index for target identity, candidate identity, followee mapping, node mapping, and edge append. Legacy corpus rewrite is disabled behind a forensic flag.

Now: the active path performs map lookups and a small synchronized edge append. A fallback scan exists only if context is missing. This does not change ranking scores; it changes the obsolete side-effect path after scores already exist.

Operation	Before	Now	Impact
Candidate lookup	Multiple full corpus scans.	Map lookup from dataset context.	Large completion latency removed.
Target/followee lookup	Additional full scans.	O(1) context lookup.	No repeated million-row reads.
Dataset-Test output	Full read/write per target.	Disabled in active path.	Avoids huge obsolete file writes.
Edge append	Small append after scans.	Small synchronized append.	Preserves graph history safely.

5.3 Recommendation Failure Semantics

Before: a crashed agent or zero-score map could still leave partial, plausible-looking output, or the workflow could wait forever for a dead agent. This was especially dangerous in million-row SVM experiments.

Change: recommendation payloads carry status, candidate type, score kind, details, and explicit errors. Runs fail closed when a required model stage dies; the UI reports a failed recommendation stage instead of accepting partial zero-score results.

Now: failure is more visible. That is sometimes emotionally worse during testing, but academically better: bad runs do not masquerade as valid recommendations.

6. Large Dataset Generation, Streaming, Sparse Models, and Memory Scheduling

6.1 User Generation Engine

Before: the generator was not a reliable million-row production tool. A large request could depend on too much in-memory state, leave temporary files, or produce output that was not fully validated.

Change: `UserGenerationEngine.java` was introduced for exact-count generation, row/schema validation, atomic temporary-file replacement, deterministic allocation, and cleanup. Self-tests include ordinary and million-row paths.

Now: the generator can produce a valid million-row corpus from suitable source corpora. The generated file being valid does not mean every recommendation algorithm can consume it cheaply; that is a separate algorithm/runtime question.

6.2 Streaming Ingestion and Bounded Delivery

Before: large text datasets were repeatedly loaded, copied, and resent. Eight nodes multiplied memory pressure and could crash before scheduling had any chance to protect the expensive model stage.

Change: `ReplayableCorpusStream`, `ReplayableTweetBatchSource`, `CorpusStreamingAgent`, `CorpusStreamRoute`, `DatasetRow`, and `TweetBatchDeliveryLedger` support replayable dataset windows, retryable batch acknowledgement, and per-user indexing. The controller can build one context and distribute bounded batches rather than retaining everything everywhere.

Now: input delivery is more predictable and observable. The node dashboard can show whether nodes are receiving records, preprocessing, waiting for peers, waiting for memory, waiting for a shared model, running a model, or complete.

6.3 Sparse ARFF and Exact SVM Worker Isolation

Before: normal SVM nodes created dense ARFF-like structures and Weka model objects in the shared JADE JVM. Eight nodes could each build large feature maps and crash the main process before any worker scheduler executed.

Change: sparse ARFF serialization, exact SVM corpus admission, corpus spooling, isolated Weka worker processes, worker result parsing, and memory leases were added. Exact SVM still uses Weka SMO; representation and scheduling changed, not the classifier family.

Now: if work fits, requested nodes can run together. If it does not fit, nodes run in safe waves or the run fails early before partial graph side effects. A single worker can still fail if the exact Weka model is intrinsically too large for its heap; scheduling reduces combined pressure, not mathematical complexity.

6.4 Adaptive Heap and Parallelism Controls

Before: heap size required launcher edits, and the program had no user-visible model-memory budget. Users could request eight nodes on a heap that could not safely hold eight model stages.

Change: `RuntimeResourceSettings`, `RuntimeRelauncher`, `RuntimeRelaunchHelper`, and the GUI `Maximum Java heap (MB)` control let the user persist a heap size and restart the app. `RecommendationMemoryCoordinator` estimates SVM, MLP, and generic model work and admits safe parallelism according to model budget, live heap, and requested node count.

Now: multi-node execution is not artificially capped; it is capacity-aware. On strong hardware with enough heap, more nodes can run concurrently. On constrained heap, work is serialized into waves. The UI shows requested/active/waiting counts and reserved memory.

Path	What changed	Recommendation impact	Speed impact
Exact SVM	Sparse input, worker isolation, heap-aware leases.	Same Weka SMO family; older SVM scoring bug separately fixed.	Can avoid crashes; speed depends on data and Weka complexity.
Scalable Linear SVM	Custom sparse linear model for high-cardinality cases.	Different algorithm, not exact Weka.	Much more scalable, but must not be compared as bare Weka SMO.
K-Means	Sparse point access and better text/load balancing.	Legacy cluster logic retained.	Multi-node benefit clearer on heavier corpora.
MLP	Model averaging and memory leases.	Same broad neural classifier path.	Avoids simultaneous large model overload, not a cheap million-row guarantee.
Doc2Vec	Streaming corpus, shared cache, mmap scoring vectors.	Same shared model/scorer for fixed cache.	Warm scoring can be very fast; cold training remains expensive.

7. Exact Multi-Node Lab, Journal Dataset, and Algorithm Correctness

7.1 Why the Exact Lab Exists

The regular simulator is a multi-agent workflow with graph side effects, user/mobile agents, edge persistence, status popups, and legacy research behavior. It is not ideal for proving that eight nodes preserve the exact same target-level answer as one node.

The Exact Multi-Node Population Lab is a separate tab that runs complete target jobs over one canonical dataset snapshot, assigns whole target users to workers, avoids graph side effects, and can verify every target fingerprint against a one-worker baseline. It is a new feature, not a replacement for the simulator.

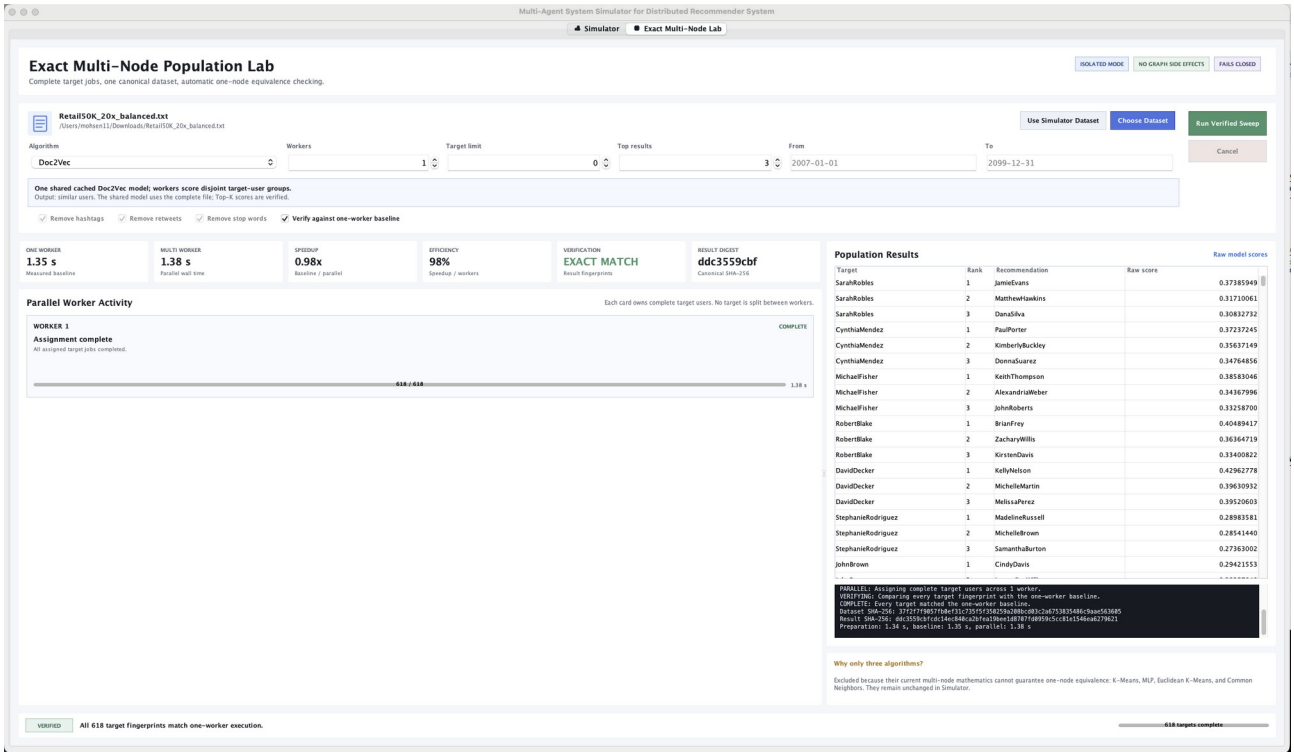


Figure 7. User-provided screenshot of the Exact Multi-Node Population Lab, including dataset choice, worker count, verification, and raw result table.

Regular Simulator batch	Exact Multi-Node Lab
Uses JADE agents, MobileAgent queries, Organizer merge, and graph edge persistence.	Runs isolated target jobs without graph side effects.
Nodes usually shard candidate/work data for a simulation.	Workers receive complete target-user jobs; no target is split between workers.
Designed for the original DSMP workflow and graph evolution.	Designed for correctness/performance experiments and one-worker equivalence.
Can use all algorithms in the simulator, including ones whose multi-node math is legacy.	Only enables algorithms where equivalence can be guaranteed for the lab design.

7.2 Exact SVM versus Scalable Linear SVM

`SVM` in the normal/exact path means Weka SMO. It is the closest path to bare Weka and is the only fair Weka comparison when configuration and workload match. `Scalable Linear SVM` is a custom sparse linear one-vs-rest model. It exists for huge high-cardinality experiments, but it is not Weka SMO and should not be reported as the same algorithm.

The report should therefore use precise wording: Exact SVM is calibrated Weka SMO; Scalable Linear SVM is a separate scalable model family. A speedup in the scalable path is useful engineering, but it does not prove Weka itself became faster.

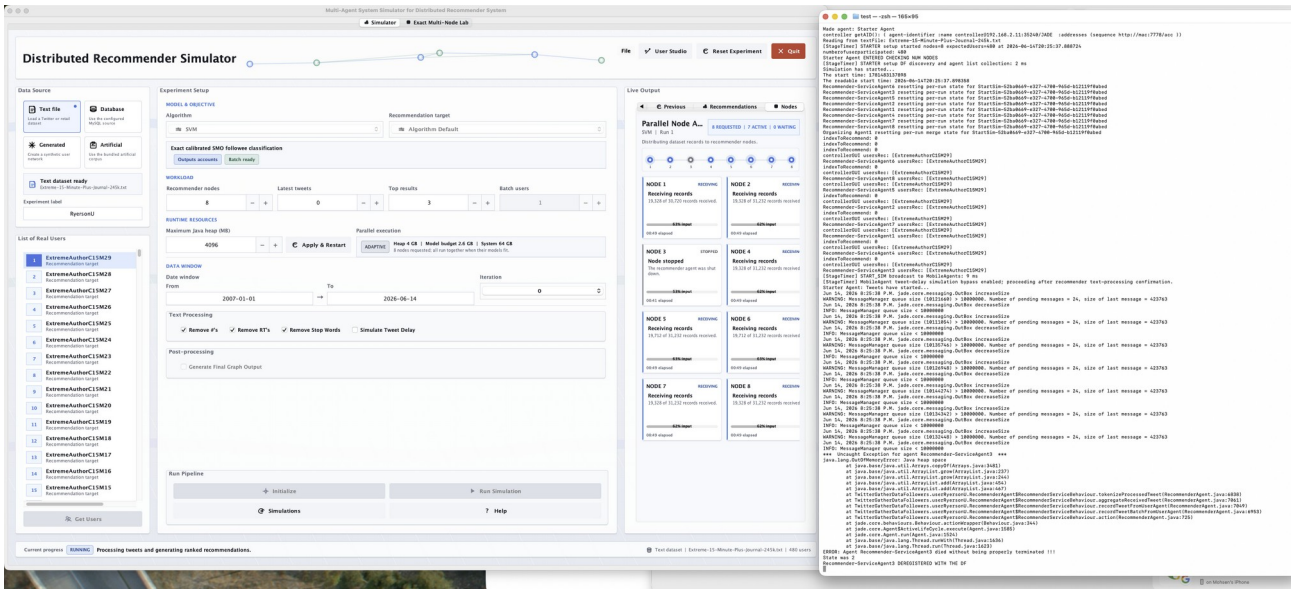


Figure 8. Development screenshot showing why fail-closed behavior matters: an exact SVM high-cardinality run can hit heap limits and must not produce partial recommendations.

7.3 Journal Dataset Findings

The professor described a journal dataset prepared in follower-followee format: authors are represented as followers/user IDs, journals are followees/classes, and paper titles are the posts/text. This matches the six-field simulator idea as long as the selected file obeys the same tab-separated schema and field semantics.

What works: Doc2Vec and SVM are conceptually appropriate for journal titles. Doc2Vec ranks similar authors from title text. SVM predicts or ranks journals/classes from title-derived user documents. The current code can load the observed 38k journal file, and smaller journal subsets such as 1k/2k/5k are present in `TestedDataset`.

What needs care: exact Weka SVM can be memory-heavy with many authors and title terms; the journal file can contain very long author-like identities and many unique terms; initializing too many individual user agents can hit OS/thread/process limits; and single-class or extremely imbalanced journal slices are not meaningful for classifier evaluation.

Journal use case	Recommended setting	Why
Similar authors from titles	Doc2Vec or Similar Users mode.	Natural output is other author/user identities.
Journal classification	SVM Algorithm Default or Followee Accounts.	Natural output is field-0 journal/followee class.
Large exact Weka run	Increase heap, use fewer nodes, or expect safe waves/fail-closed behavior.	Weka SMO is memory intensive with wide vocabularies.
Demonstrating parallelism	Exact Lab with verification enabled.	Separates target-level correctness from graph side effects.

7.4 Algorithm-by-algorithm Correctness Verdict

Algorithm	Current verdict	Why it is correct or limited
Similarity	Correct for user-ranking default.	Sparse cosine scoring ranks users; converted account mode is a clear mapping from similar users to accounts.
K-Means	Correct as legacy cluster ranking, not exact-lab supported.	Default path retains legacy K-Means scoring; multi-node exact equivalence is not guaranteed enough for Exact Lab.
K-Means Euclidean	Same as K-Means with Euclidean distance.	Works in simulator but remains legacy/expensive on very wide corpora.
SVM	Corrected Weka SMO path.	Retail parsing, stratified split, ARFF quoting, calibrated distributions, and memory scheduling address known defects.
MLP	Works for supported sizes; high risk on very wide corpora.	Neuroph path remains dense and memory-heavy; scheduling reduces crashes but not intrinsic network size.
Doc2Vec	Correct for fixed-cache target scoring.	Batch uses same shard scorer per target; cache transparency now explains warm scoring.
Scalable Linear SVM	Useful separate mode, not Weka SMO.	Fast sparse linear classifier; must be reported as a different algorithm.
Common Neighbors	Not considered a working modern recommendation implementation.	Excluded from Exact Lab and should not be used as a supported algorithm claim.

8. Verification Record and Risk Register

8.1 Current Verification

Check	Result	Meaning
Production Java compile	Passed with deprecation/unchecked notes.	Current `userRyersonU/*.java` compiles against project libraries.
PopulationSweepPanelSelfTest	Passed and wrote PNG.	Exact Lab panel renders nonblank in headless test.
NodeExecutionDashboardSelfTest	Passed and wrote PNG.	Parallel node dashboard renders nonblank with eight seeded states.
RecommendationScoreMergerSelfTest	Passed.	Typed score merge, target modes, conversion, normalization, and payload serialization behave.
RecommendationPostProcessorSelfTest	Passed.	Fast post-processing writes/accumulates edges and handles no-candidate cases.
RuntimeResourceSettingsSelfTest	Passed.	Heap setting validation, persistence, and relaunch command replacement behave.
Doc2VecBatchRunnerSelfTest	Passed with known Gensim diagnostic.	Structured batch output parsing and unsafe diagnostic detection work.

The current compile command used for this report was: `javac -nowarn -cp "lib/*" -d /private/tmp/dsmp-current-report-compile TwitterGatherDataFollowers/userRyersonU/*.java`.

8.2 Risk Register

Risk	Severity	Control	Next action
Dirty worktree mixes source and generated artifacts.	Critical	This report names delivery state.	Prepare source-only commits from an allow-list.
Converted target modes misreported as native algorithms.	High	Mode labels and typed payloads.	Use `Algorithm Default` for native claims.
Doc2Vec warm cache mistaken for skipped work.	High	Cache transparency panels and remove button.	Record cache key in experiments.
Exact Weka SVM still OOMs on extreme feature spaces.	High	Admission, spooling, worker isolation, fail-closed errors.	Benchmark per corpus and avoid impossible heaps.
Python Doc2Vec memory outside Java heap.	Medium	Memory-mapped vectors and visible cache state.	Add process RSS telemetry later.
K-Means/MLP remain costly on wide corpora.	Medium	Sparse support and model leases.	Run algorithm-specific scale matrix before claims.
Graph communities differ from professor expectation.	Medium	Reset, edge accumulation, target-mode clarity.	Compare edge distribution and Gephi settings.
Uncommitted UI lacks broad regression tests.	Medium	Headless render tests for major panels.	Add component state tests for button enable/disable.

8.3 Recommendation Integrity Summary

Change	Can it change recommendations?	Explanation
GUI layout, picker, status bar, screenshots	No	Presentation/input ergonomics only.
Doc2Vec batch for fixed cache	No intended per-target change	Same scorer; now repeated correctly for every target.
Retail parser and ARFF quoting	Yes, as bug fix	Old behavior corrupted class names with spaces.
SVM stratified split and distributions	Yes, as bug fix	Old SVM could train with missing classes and hard 1/0 scores.
Target modes	Yes, intentionally	Changing output entity type changes the recommendation objective.
Graph checkbox	Ranked list no	Only skips visualization/export; edge side effect remains.
Post-processing rewrite removal	Ranked list no	Changes side-effect speed, not model score calculation.
Memory scheduling	No intended score change	Changes when work runs and whether it fails safely.

Scalable Linear SVM	Yes	It is a new separate model, not exact Weka SVM.

9. Detailed Per-change Ledger

This section is intentionally repetitive because it is meant to be useful during academic review. Each item records what the system did before, why that behavior was unsafe or weak, what was changed, how the current program behaves, and what risk remains.

9.1 Dataset and Format Corrections

9.1.1 Retail50K class names with spaces

Lens	Details
Before	Tweet fields were encoded into one space-delimited JADE message.
Why this was bad	`T&T Supermarket` was read as `T&T`, which broke class lookup and could crash MLP or corrupt SVM labels.
What changed	The active message path became tab-delimited and the receiver parses tabs before falling back to legacy space parsing.
Current behavior	Class/followee names with spaces are preserved exactly through the agent protocol.
Efficiency / correctness effect	This is a correctness fix; it does not intentionally make the algorithm faster, but it prevents invalid training labels.
Residual risk	A future non-tab-safe payload format could reintroduce the issue. Keep tests with a class containing spaces.

9.1.2 ARFF quoting for Weka

Lens	Details
Before	Class attribute values and row labels were written raw into ARFF files.
Why this was bad	ARFF requires quoting for values containing spaces or special characters; Weka could split or misread labels.
What changed	A class-value formatter now quotes and escapes strings used in Weka ARFF output.
Current behavior	Weka receives one valid class token for names such as `T&T Supermarket`.
Efficiency / correctness effect	This changes only file validity and label integrity. It does not replace Weka or change TF-IDF inputs.

Residual risk	Any new ARFF writer must reuse the same escaping rule; otherwise a parallel path can drift.

9.1.3 Selected uploaded file path

Lens	Details
Before	Some downstream code assumed the selected dataset had been copied into `important-stuff/`.
Why this was bad	A valid upload from Downloads could later fail with `FileNotFoundException` because the wrong path was opened.
What changed	The code now carries and uses the selected `File` object for corpus reads and post-processing.
Current behavior	Uploaded datasets can be used from their actual selected location.
Efficiency / correctness effect	The improvement removes false missing-file failures and avoids unnecessary copying.
Residual risk	External deletion or movement of the selected file during a run can still break the run.

9.1.4 Dataset-specific graph files

Lens	Details
Before	Graph edges were written to shared generic filenames.
Why this was bad	Old edges from a previous dataset could appear in a new dataset graph, invalidating community statistics.
What changed	Graph edge and GML filenames are derived from the selected dataset base name and target mode where relevant.
Current behavior	Retail, Twitter, journal, and generated datasets can maintain separate accumulated edge files.
Efficiency / correctness effect	Graph results become interpretable because edge history belongs to the intended dataset.
Residual risk	If two different files share the same base name in different folders, a stronger canonical naming scheme would be safer.

9.1.5 Auto name-number mapping

Lens	Details

Before	Retail50K expected a manually prepared `name-number-Retail50K.txt` mapping.
Why this was bad	Missing mapping files caused incomplete graph output or old node counts.
What changed	The program can generate the mapping from the selected dataset when needed.
Current behavior	The graph has a stable node map based on the actual user/followee identities in the dataset.
Efficiency / correctness effect	This makes uploaded datasets much easier to run and reduces manual setup error.
Residual risk	Very large mappings should still be built atomically and cached, which later work reinforces.

9.2 Algorithmic Recommendation Corrections

9.2.1 SVM class-preserving train/test split

Lens	Details
Before	The old split built the test set first and could consume all examples from minority classes.
Why this was bad	If a class has zero training examples, Weka cannot learn that class and recommendations collapse toward larger classes.
What changed	The split now partitions users inside each class and keeps at least one training example when possible.
Current behavior	Small Retail classes such as `T&T Supermarket` are no longer silently removed from training.
Efficiency / correctness effect	This is a correctness repair that makes SVM recommendations more meaningful on imbalanced user-level classes.
Residual risk	A class with only one total user still cannot support both train and test examples; the report must disclose class counts.

9.2.2 SVM calibrated score distributions

Lens	Details
Before	SVM recommendation scores were produced from one hard predicted class.

Why this was bad	The result became 100/0 style output, which was not a useful ranked score distribution.
What changed	SMO calibration is enabled and `distributionForInstance()` is converted into a score map.
Current behavior	SVM returns a ranked map across followee accounts instead of only one hard class.
Efficiency / correctness effect	Recommendation output is smoother and more informative while retaining Weka SMO.
Residual risk	Calibrated probabilities depend on Weka behavior and data quality; they are still normalized later for display.

9.2.3 SVM batch mode

Lens	Details
Before	Single-user SVM was too fast for node overhead to show obvious speedup, and manual repeated runs paid fixed overhead every time.
Why this was bad	Batching many target users manually was slow and hard to reproduce.
What changed	SVM batch mode selects multiple target users and trains/scores each target independently.
Current behavior	Each target receives its own SVM score map; batch targets are excluded from training where appropriate.
Efficiency / correctness effect	Throughput improves by amortizing fixed simulator cost over multiple targets.
Residual risk	The regular simulator still has graph side effects and random edge persistence, so batch graph output is not identical to manual runs unless those conditions are controlled.

9.2.4 Doc2Vec shared vector space

Lens	Details
Before	Earlier multi-node-looking Doc2Vec designs could train separate models per node.
Why this was bad	Independent Doc2Vec models do not share a comparable vector space, so merging their scores is mathematically unsafe.
What changed	The active path trains or loads one shared model and lets nodes score shards against that shared representation.
Current behavior	Doc2Vec node scores are comparable because they come from the same vector space.

Efficiency / correctness effect	Correctness improves dramatically; repeated warm runs also avoid retraining.
Residual risk	Fresh multi-worker training is not bit-for-bit deterministic; fixed-cache comparisons are the honest equivalence test.

9.2.5 Doc2Vec regular batch mode

Lens	Details
Before	The Java branch passed only the first selected target to Python and copied that target's scores to every target.
Why this was bad	This was a silent false result: the GUI could show many targets even though only one was actually scored.
What changed	A structured batch runner now scores each target independently through the same shard scorer.
Current behavior	Doc2Vec batch users receive target-specific recommendation maps.
Efficiency / correctness effect	Python/model load overhead is reduced because one process can score many targets.
Residual risk	A missing target remains an explicit no-candidate/error state; the UI should not hide it.

9.2.6 Recommendation target modes

Lens	Details
Before	The UI did not explain whether an algorithm recommended users or accounts.
Why this was bad	Users could compare Doc2Vec people to SVM retailers and think one algorithm was misimplemented.
What changed	The GUI and payloads now distinguish `Algorithm Default`, `Similar Users`, and `Followee Accounts`.
Current behavior	The chosen recommendation objective is visible before the run and in result files/cards.
Efficiency / correctness effect	This prevents semantic confusion and enables deliberate cross-type recommendation experiments.
Residual risk	Converted modes intentionally change the recommendation objective and must not be described as native algorithm output.

9.2.7 Score normalization explanation

Lens	Details
Before	Top-ranked scores appeared as 100 for every target without explanation.
Why this was bad	Users could misread 100 as absolute confidence rather than relative maximum after normalization.
What changed	The report and GUI design treat these as relative model/ranking scores and preserve raw-score access where relevant.
Current behavior	A top item at 100 means it is the maximum positive score for that target after merge normalization.
Efficiency / correctness effect	This improves interpretation without changing ranking calculations.
Residual risk	The visible label `MODEL SCORE` can still be misunderstood; `Relative Score` would be even clearer in a future UI pass.

9.3 GUI and Workflow Changes

9.3.1 Modern three-column simulator UI

Lens	Details
Before	The legacy UI placed many controls in a dense form and showed recommendations as raw text.
Why this was bad	Important choices were hard to discover and error states looked like normal console noise.
What changed	The workspace was reorganized into data source/user list, experiment setup, and live output columns.
Current behavior	Users see source selection, model objective, workload, runtime resources, text processing, post-processing, and results as separate groups.
Efficiency / correctness effect	No recommendation math changes; the benefit is reduced operator error and faster diagnosis.
Residual risk	Swing visual regression testing remains lightweight; new screenshots should be captured before releases.

9.3.2 Recommendation cards

Lens	Details

Before	The recommendation panel printed equals signs and text lines.
Why this was bad	Batch output became ugly and hard to scan, especially for dozens or hundreds of targets.
What changed	Typed result payloads now render per-target cards with rank, candidate, score, and detail.
Current behavior	The GUI clearly shows target count, match count, and no-candidate cases.
Efficiency / correctness effect	This is presentation only; score order comes from the merged payload.
Residual risk	If payload construction fails, raw console diagnostics still matter for debugging.

9.3.3 Searchable file picker

Lens	Details
Before	The native `JFileChooser` crashed on macOS under the decorated dialog state and had limited search help.
Why this was bad	Selecting a dataset could throw an AWT exception before the simulation even started.
What changed	`ModernFilePicker` provides an app-owned searchable picker.
Current behavior	Users can locate dataset files more reliably and avoid the decorated dialog exception.
Efficiency / correctness effect	Workflow reliability improves; algorithm behavior does not change.
Residual risk	Native file-management features are not all replicated; very large directories may need additional indexing later.

9.3.4 Initialize button state

Lens	Details
Before	Initialize could stay disabled even after users were loaded.
Why this was bad	The user could not start a valid run despite having selected a target.
What changed	Button-state logic was tightened around source selection, user loading, and run lifecycle.
Current behavior	Initialize becomes available when the current dataset/user state is valid.
Efficiency / correctness effect	This removes a GUI blocker without changing simulation configuration.

Residual risk	Button state must continue to be tested after any GUI layout refactor.

9.3.5 Bottom status bar

Lens	Details
Before	Status text was vague and the status pill appeared before `Current progress`.
Why this was bad	Users could not tell whether the app was loading users, processing features, merging, graphing, or complete.
What changed	The status bar now places the phase in front of the current-progress message and includes dataset/user context.
Current behavior	The user can see what stage the simulator is in at each moment.
Efficiency / correctness effect	No scoring changes; reduces false stuck reports.
Residual risk	Some legacy console output still remains noisy and could be further structured.

9.3.6 User Generation Studio branding

Lens	Details
Before	The studio header had colorful mode badges under the title and lacked academic attribution placement.
Why this was bad	The visual hierarchy looked misaligned and did not credit the lab cleanly.
What changed	The header was aligned left, badges removed, and Dr. Abdolreza Abhari plus Mohsen Alavi attribution added in a compact lab identity area.
Current behavior	The first page looks more professional and communicates ownership/support without clutter.
Efficiency / correctness effect	Visual/presentation change only.
Residual risk	Attribution wording should be approved by the professor before formal publication.

9.3.7 Doc2Vec cache UI

Lens	Details
Before	A cached Doc2Vec run could finish extremely fast with no explanation.

Why this was bad	Users reasonably suspected the algorithm had skipped work or returned invalid results.
What changed	Both Simulator and Exact Lab tabs now inspect and display cache state, rows, users, vectors, cache size, and key.
Current behavior	A warm-cache run is visible as scoring-only; a cold-cache run warns that training will happen.
Efficiency / correctness effect	Improves trust and experiment reproducibility without changing Doc2Vec scores.
Residual risk	Cache inspection itself depends on filesystem metadata and should be refreshed after external file edits.

9.4 Graph and Experiment Lifecycle

9.4.1 Explicit Reset Experiment

Lens	Details
Before	New experiments relied on closing the GUI or hidden cleanup attached to Get Users.
Why this was bad	Old graph and recommendation files could contaminate the next dataset, and cleanup timing was confusing.
What changed	A dedicated Reset Experiment action stops agents, clears state, and removes known transient outputs.
Current behavior	Users can start a clean experiment without closing the app and without deleting source datasets or caches.
Efficiency / correctness effect	Improves reproducibility and prevents stale-edge contamination.
Residual risk	Any new artifact type must be added to the reset inventory.

9.4.2 Graph output checkbox

Lens	Details
Before	Every successful recommendation entered Gephi visualization/export.
Why this was bad	Graph rendering made runs feel stuck and was unnecessary when only recommendations were needed.
What changed	`Generate Final Graph Output` controls the final Gephi display/export phase.

Current behavior	Unchecked runs still persist nodes and edges; later checked runs render accumulated graph state.
Efficiency / correctness effect	Saves time while preserving experimental graph history.
Residual risk	Users must understand that graph disabled does not mean edge disabled.

9.4.3 Run identifiers and stale message protection

Lens	Details
Before	Long-lived agents could receive or act on messages from a previous run.
Why this was bad	A new run could hang, merge wrong scores, or finish based on old completion messages.
What changed	`RecommendationRunLifecycle` tags and classifies run messages.
Current behavior	Agents accept only current-run data and reject stale or retired messages.
Efficiency / correctness effect	Reliability improves without changing score formulas.
Residual risk	The retired-run history is finite; extremely delayed messages beyond the retained window need monitoring.

9.4.4 Idle CPU and Gephi cleanup

Lens	Details
Before	Agents kept polling and the embedded Gephi project could remain open after export.
Why this was bad	The app could consume very high CPU/RAM hours after the visible work completed.
What changed	Idle handlers use blocking receives/templates, popups are asynchronous, and Gephi projects are closed after export.
Current behavior	Leaving the GUI open should no longer keep expensive simulation loops active.
Efficiency / correctness effect	Reduces idle CPU and memory retention.
Residual risk	External native libraries and macOS graphics resources can still consume memory during actual graph rendering.

9.5 Scalability and Parallelism

9.5.1 Million-row generation

Lens	Details
Before	The generator was not designed as a validated, atomic million-row producer.
Why this was bad	Large generated files could be malformed, partial, or hard to trust.
What changed	`UserGenerationEngine` performs exact allocation, streaming writes, validation, and atomic completion.
Current behavior	A valid million-row file can be generated from suitable corpora.
Efficiency / correctness effect	Generation became reliable; recommendation algorithms still have their own limits.
Residual risk	A generated corpus is synthetic and should not be treated as new real-world evidence.

9.5.2 Streaming dataset intake

Lens	Details
Before	Large corpora were repeatedly read and copied into many in-memory structures.
Why this was bad	Eight nodes multiplied memory pressure before model scheduling could help.
What changed	Replayable streams, batch sources, corpus routes, and context indexes reduce duplication.
Current behavior	The simulator can ingest large text datasets more predictably.
Efficiency / correctness effect	Improves memory behavior and creates reusable context for fast post-processing.
Residual risk	Very high user counts can still create many agents or large indexes; no representation removes all cost.

9.5.3 Sparse ARFF and Weka workers

Lens	Details
Before	Dense ARFF construction and Weka models lived in the shared main JVM.
Why this was bad	SVM could throw heap errors before any worker scheduler had a chance to limit concurrency.

What changed	Sparse ARFF writing, exact SVM corpus spooling, isolated worker JVMs, and worker result parsing were added.
Current behavior	Exact SVM can run safer waves and fail closed when a corpus cannot fit.
Efficiency / correctness effect	Memory pressure is reduced and partial invalid recommendations are prevented.
Residual risk	A single Weka SMO model can still be too large; this is a known algorithmic limit.

9.5.4 Adaptive memory leases

Lens	Details
Before	The program either launched all requested nodes or used crude caps.
Why this was bad	All-at-once could crash; fixed caps wasted capable hardware.
What changed	`RecommendationMemoryCoordinator` estimates model work and admits safe concurrent leases.
Current behavior	If eight nodes fit, eight can run. If not, the rest wait and run in waves.
Efficiency / correctness effect	Preserves multi-node speed potential while improving reliability.
Residual risk	Estimates are heuristics; unusual datasets can underpredict actual Weka/Neuroph memory.

9.5.5 Heap GUI and restart

Lens	Details
Before	Changing max heap required editing launch scripts or environment variables.
Why this was bad	Users could not easily test whether more heap allowed more concurrent nodes.
What changed	The GUI exposes max heap in MB and relaunches with updated `-Xmx` through runtime settings.
Current behavior	Users can increase heap, restart, and let the adaptive scheduler use the new budget.
Efficiency / correctness effect	Makes resource experiments transparent and repeatable.
Residual risk	Excessive heap can cause OS swapping; the GUI warning is guidance, not a guarantee.

9.5.6 Exact Multi-Node Population Lab

Lens	Details
Before	Regular simulator runs mixed parallelism with graph effects, random persisted edges, and legacy agent behavior.
Why this was bad	It was hard to prove that node count changed speed without changing target-level results.
What changed	The Exact Lab runs isolated target jobs, assigns whole targets to workers, and verifies against a one-worker baseline.
Current behavior	Users can demonstrate parallel effects while checking exact output equivalence for supported algorithms.
Efficiency / correctness effect	This is the clearest speed/correctness demonstration feature.
Residual risk	It intentionally excludes algorithms whose current multi-node mathematics cannot guarantee one-worker equivalence.

9.5.7 Real-time node dashboard

Lens	Details
Before	The user could see console lines but not a structured view of each requested node.
Why this was bad	Parallel execution was invisible, so waiting looked like a hang.
What changed	`NodeExecutionMonitor` and `NodeExecutionDashboard` show node states such as receiving, preprocessing, waiting, running, complete, failed, or stopped.
Current behavior	The GUI can explain whether all requested nodes are working, waiting for memory, or waiting for a shared model.
Efficiency / correctness effect	Improves observability and debugging without changing algorithms.
Residual risk	The dashboard depends on agents reporting state consistently; new stages should add monitor events.

9.6 Dataset Interpretation and Communication

9.6.1 Retail versus Twitter semantics

Lens	Details

Before	The simulator accepted any six-field file but did not explain what field 0 meant.
Why this was bad	Retail SVM output looked like store names while Doc2Vec output looked like people, causing confusion.
What changed	The report and GUI now explain that SVM/MLP classify field-0 accounts while Doc2Vec/user rankers rank user identities.
Current behavior	Users can choose the output entity type or explain native defaults correctly.
Efficiency / correctness effect	Prevents false bug reports about an intentional data-semantics difference.
Residual risk	Each new corpus still needs a short schema note before results are interpreted academically.

9.6.2 Journal dataset interpretation

Lens	Details
Before	The journal dataset was not initially identified in the current workflow.
Why this was bad	Users could treat journal results like Retail/Twitter without understanding author/journal/title semantics.
What changed	The dataset was analyzed as author user IDs, journal followees/classes, and paper titles as posts.
Current behavior	Doc2Vec can rank similar authors; SVM can classify/rank journals from title-derived documents.
Efficiency / correctness effect	Clarifies professor's intended research use and avoids misreporting output type.
Residual risk	Large journal files remain memory-sensitive, especially for exact Weka SVM.

9.6.3 Professor communication drafts

Lens	Details
Before	Questions about SVM/Doc2Vec output semantics and journal data were unresolved.
Why this was bad	Without clarification, the implementation could optimize the wrong research objective.
What changed	Emails were drafted to ask about intended output entity and to report journal-dataset findings.
Current behavior	Communication artifacts support academic alignment but are not code changes.

Efficiency / correctness effect	Improves project governance.
Residual risk	Final wording and claims should be reviewed before sending or using in thesis/paper text.

Appendix A. File-level Map

File or class	Role
ControllerAgentGui.java	Modern GUI, source cards, setup panels, recommendation cards, heap controls, cache UI, reset, lab tabs.
ControllerAgent.java	Dataset loading, context, agent startup, reset, SVM split support, runtime coordination.
RecommenderAgent.java	Algorithm paths, Doc2Vec/SVM batch, sparse preparation, node monitor updates, model leases.
MobileAgent.java	Tweet/query flow, blocking receive, result display, post-processing request construction.
StarterAgent.java	Run orchestration, optional graph output, cleanup, completion state.
OrganizingAgent.java	Score aggregation, typed payload merge, MLP averaging handoff.
Doc2VecBatchRunner.java / doc2vec_batch.py	Structured multi-target Doc2Vec bridge.
doc2vec_shared.py	Shared model training, cache signature, vector cache, shard scorer.
Doc2VecCacheAdmin.java / doc2vec_cache_admin.py	Cache inspection/deletion for UI transparency.
Recommendation* classes	Target modes, score payloads, candidate evidence, merge policy, run lifecycle, memory coordination.
UserGenerationEngine.java	Validated exact-count synthetic dataset generation.
Replayable* and CorpusStreamingAgent	Large dataset streaming, retryable bounded delivery, per-run batch state.
SparseArffWriter.java	Sparse Weka ARFF serialization.
ExactPopulationSweepEngine.java / PopulationSweepPanel.java	Isolated multi-worker population lab and GUI.
NodeExecutionMonitor.java / NodeExecutionDashboard.java	Real-time node state telemetry and panel rendering.
RuntimeResourceSettings.java and relaunch classes	Heap persistence and app restart.
ModernFilePicker.java	Searchable robust dataset picker.

Appendix B. Timeline

Stage	Work completed
Before this chat / June 7 report	Retail50K parsing, ARFF quoting, graph mapping, timing instrumentation, K-Means speed investigation, SVM split/scoring repair.
Doc2Vec batch and reset	Correct batch scoring and explicit experiment reset; committed and pushed earlier.
GUI wave	Modern simulator UI, recommendation cards, source/setup/status redesign, file picker, user studio attribution.
Semantics wave	Recommendation target modes and explanation of SVM/Doc2Vec entity mismatch.
Graph/lifecycle wave	Graph checkbox, edge persistence rule, idle CPU cleanup, blocking receive, stale-run isolation.
Scale wave	Million-row generation, streaming, sparse ARFF, adaptive memory, heap GUI.
Exact lab wave	Population sweep tab, node dashboard, one-worker equivalence checks.
Journal wave	Professor dataset analysis and current behavior with author-title-journal data.
Cache transparency wave	Doc2Vec warm/cold/stale cache cards in Exact Lab and Simulator, scoped cache removal.

Appendix C. Practical Operating Guidance

- Use `Algorithm Default` when comparing with historical simulator behavior.
- Use `Similar Users` or `Followee Accounts` only when the research question explicitly asks for that entity type.
- For Doc2Vec timing, always state whether the cache was warm or cold and record the cache key.
- For SVM/Weka experiments, record heap, node count, dataset row count, user count, feature count, and class distribution.
- Keep graph output unchecked when only recommendations are needed; check it later to render accumulated edges.
- Use Reset Experiment before starting a truly new dataset experiment.
- Do not use partial results after a fail-closed OOM or worker failure.
- Treat Scalable Linear SVM as a separate algorithm from exact Weka SMO.
- For journal data, describe whether the output is similar authors or journal/followee classes.
- Before sharing or committing, clean generated corpora, ARFF files, caches, logs, screenshots, class files, and `.DS_Store` artifacts.

Final Assessment

Across the full workstream, the simulator moved from a fragile research prototype that could silently mix entity types, reuse stale graph state, overload memory, and hide why a run was fast, into a much clearer experimental platform. The strongest improvements are not only speed. They are honesty: the GUI now has places to show what is selected, what is cached, what type of recommendation is being produced, what each node is doing, and whether a failure invalidates the run.

The remaining task is release discipline. The code should be split into clean commits, generated artifacts should be excluded, and a clean clone should reproduce compile and focused tests. Once that is done, the project will be in a strong position for professor review and for defensible performance/correctness experiments.